Understanding Modern AI: A Guide for Beginners

This document explains the technical pillars of the **Elite Estate** platform: **AI Semantic Search**, **Embedding Models**, **RAG**, and **n8n Automation**. These concepts are what elevate the project to a sophisticated, intelligent system.

---

1. AI Semantic Search (The "Global Map" Metaphor)

The Old Way: Keyword Search

Traditional search works like a dictionary. If you search for "House with a pool," the computer looks for those *exact words*. If a listing says "Villa with a swimming area," the computer might miss it because the words don't match, even though the meaning is the same.

The New Way: AI Semantic Search

Semantic search doesn't look for words; it looks for **meaning**.

Metaphor: The Global Map
Imagine every idea in the world is a location on a giant map.

- "House" and "Villa" are different words, but on the map, they are parked right next to each other.
- "Swimming pool" and "Ocean view" are both related to water, so they are in the same "Waterfront" neighborhood.

The "Translator": Embedding Models

If Semantic Search is the "Map," the **Embedding Model** is the translator that helps the computer build that map.

Why we use it:
Computers are great at math (numbers), but they don't actually understand human language (words). To help a computer "understand" a property description, we need to turn that text into a long list of numbers.

How it works:

1. We give the model a sentence: *"Luxury villa with a large garden."*
2. The model looks at millions of examples it has learned and assigns a numerical value to every concept in that sentence.
3. The result is a **Vector** (a list of 768 numbers). These numbers are like the **DNA** of that sentence. If two sentences have similar "DNA" (similar numbers), the computer knows they have a similar meaning.

How it works in practice:

1. When we add a property, the AI gives it "GPS coordinates" (called **Embeddings**).
2. When a user searches for something, the AI plots their query on the same map.
3. The system then simply looks for the properties that are **closest** to the user's point on the map.

---

2. RAG: Retrieval-Augmented Generation (The "Open-Book Exam" Metaphor)

The Problem: The "Know-it-all" AI

Standard AI models (like ChatGPT or Gemini) were trained on a massive amount of data, but they don't know the *specific* details of your private business. If you ask them a specific question they don't know, they might "hallucinate" (confidently make up a fake answer).

The Solution: RAG (The "Open-Book Exam")

RAG ensures the AI remains truthful by giving it a specific set of documents to read before it answers.

Metaphor: The Open-Book Exam

- **Standard AI**: Like a student trying to pass a test purely from memory.
- **RAG AI**: Like a student taking an **Open-Book Exam**. We give the student a "textbook" (our property data) and say: *"Answer the user's question, but ONLY using the information in this textbook."*

---

3. n8n: The "Digital Orchestrator"

What is n8n?

In a complex project, you have many different "workers": a Database, an AI Model, Telegram, and Google Calendar. Normally, they don't know how to talk to each other. **n8n** is the **Workflow Automation** tool that acts as the supervisor, connecting everyone together.

How we are using it

n8n is the "brain" behind our automation. We use it to create **active workflows**:

1. **The Telegram Bridge**: When a user sends a message on Telegram, n8n catches it, sends it to the AI for a response, and then sends that response back to the user.
2. **The Calendar Sync**: When a visit is booked, n8n automatically reaches out to Google Calendar to create the event without any human intervention.
3. **Smart Reminders**: n8n has a clock that "wakes up" every hour. It checks the database for upcoming visits and sends a "Don't forget!" message to the client on Telegram automatically.

Why it's better than custom code:
Instead of writing thousands of lines of code to connect these apps, we use n8n's visual interface to build a clear, logical flowchart. This makes the system more stable, easier to monitor, and much faster to build.

---

4. Summary

- **Semantic Search** is about **understanding what the user wants**.
- **Embedding Models** are the **translators** that turn words into numbers the computer can map.
- **RAG** is about **ensuring the AI tells the truth** by making it read our specific data.
- **n8n** is the **glue** that connects all these different services into one automated platform.

Elite Estate: Local Setup Guide

Follow these steps to get the full Elite Estate stack running on your local machine.

Prerequisites

Ensure you have the following installed:

- **Docker & Docker Compose**: For running the entire stack.
- **Ollama**: For local AI embeddings (runs on your host machine).
- **ngrok account**: Required for Telegram bot webhooks and secure external access.

---

1. Local AI Embeddings (Ollama)

Ollama runs on your host machine to leverage your system's performance for generating vector embeddings.

1. **Install Ollama**: Download from ollama.com (https://ollama.com/).
2. **Install Model**: Run the following in your terminal to download the embedding model:

   ```
   ollama pull nomic-embed-text
   ```

3. **Cross-Container Access**:
   - On Windows, ensure Ollama is running in the background.
   - If you encounter "Connection Refused" errors from the backend, set the system environment variable `OLLAMA_HOST=0.0.0.0` and restart the Ollama application.

---

2. Setting up n8n & External Access (ngrok)

To allow Telegram to communicate with your local n8n instance and access the n8n editor via the web, you must set up a public tunnel.

Step A: Create an ngrok Free Domain

1. Sign up at ngrok.com (https://ngrok.com/).
2. Navigate to **Cloud Edge > Domains**.
3. Click **Create Domain** (one static domain is free per account).
4. Copy your domain (e.g., `your-name.ngrok-free.app`).

Step B: Configure n8n in .env

1. Retrieve your `NGROK_AUTHTOKEN` from the ngrok dashboard.
2. Update your `.env` file with these values:

```
NGROK_AUTHTOKEN=your_token_here
N8N_HOST=your-name.ngrok-free.app
WEBHOOK_URL=https://your-name.ngrok-free.app
N8N_EDITOR_BASE_URL=https://your-name.ngrok-free.app
```

3. **Note**: Once the stack is running, you will access the n8n dashboard using your ngrok link: `https://your-name.ngrok-free.app`.

---

3. External Services Configuration

3.1 ImageKit.io (Media Management)

ImageKit handles property images via a global CDN.

1. Register at imagekit.io (https://imagekit.io/).
2. In the dashboard, go to **Developer Options**.
3. Copy the following into your `.env`:
    - `IMAGEKIT_PUBLIC_KEY`
    - `IMAGEKIT_PRIVATE_KEY`
    - `IMAGEKIT_URL_ENDPOINT` (e.g., `https://ik.imagekit.io/your_id/`)

3.2 Google Cloud Console (Calendar API)

Required for synchronizing property visits with agent calendars.

1. Go to Google Cloud Console (https://console.cloud.google.com/).
2. **Create a Project**: Select "New Project" and give it a name.
3. **Enable API**: Search for "Google Calendar API" and click **Enable**.
4. **OAuth Consent Screen**:
    - Go to **APIs & Services > OAuth consent screen**.
    - Choose "External" and fill in the required app information and support email.
5. **Create Credentials**:
    - Go to **APIs & Services > Credentials**.
    - Click **Create Credentials > OAuth client ID**.
    - Application type: **Web application**.
    - **Authorized Redirect URIs**: Add `https://your-name.ngrok-free.app/rest/oauth2-callback`.
6. Copy the **Client ID** and **Client Secret** (you will use these in the next step).

3.3 Connecting Google Calendar in n8n

Once n8n is running, you must link it to your Google account.

1. Open your n8n editor (`https://your-name.ngrok-free.app`).
2. Open the **Smart Agent** or **Reminder** workflow.
3. Click on any **Google Calendar node**.
4. In the **Credential** dropdown, select **Create New Credential**.
5. Authentication Method: Select **OAuth2**.
6. Enter the **Client ID** and **Client Secret** from the Google Cloud Console.
7. Click **Sign in with Google** and authorize the application.

3.4 OpenRouter (AI Agent)

1. Sign up at openrouter.ai (https://openrouter.ai/).
2. Create an API Key and add it to `.env` as `OPENROUTER_API_KEY`.
3. In the n8n **Smart Agent** workflow, ensure the model is set to `google/gemini-2.0-flash-001`.

3.5 Telegram Bot

1. Message @BotFather (https://t.me/botfather) to create a new bot.
2. Copy the **API Token** into your `.env`.

---

4. Launching the Application

1. **Prepare Environment**:

```
cp .env.example .env
# Ensure all keys from the steps above are filled in.
```

2. **Start the Stack**:

```
docker-compose up -d --build
```

3. **Initialize the Database**:

```
docker exec -it FastAPI_backend python seed.py
```

Access Points:

- **Frontend Dashboard**: http://localhost:3000 (http://localhost:3000)
- **Interactive API Documentation**: http://localhost:8000/docs (http://localhost:8000/docs)
- **n8n Automation Editor**: `https://your-name.ngrok-free.app` (Using your ngrok tunnel)

Troubleshooting

- **Ollama Error**: Run `ollama list` to verify `nomic-embed-text` is installed.
- **Telegram Webhook**: If the bot doesn't respond, check the `WEBHOOK_URL` in `.env` and ensure it uses `https://`.
- **Google OAuth**: Ensure the redirect URI in Google Console exactly matches the one in your `.env` (including the `/rest/oauth2-callback` suffix).

PFE Technical Documentation

AI-Driven Real Estate Automation Platform — *Elite Estate*

Project Type: End-of-Study Project (Projet de Fin d'Études — PFE)
Institution: ISET
Platform Name: Elite Estate
Academic Year: 2025–2026
Language: English

Table of Contents

---

1. Project Overview & Context

Elite Estate is a full-stack, AI-powered real estate management and automation platform developed as an end-of-study project (PFE). The project was built collaboratively by a two-person team, with each member owning a distinct technical domain:

Team Member	Domain
Member A	Full-stack platform engineering (Backend, Frontend, Database, DevOps)

Member B	AI automation and workflow engineering (n8n, Telegram, Google Calendar, RAG)

The platform is designed to digitize and automate the full lifecycle of a real estate agency: from property listing and client discovery, through visit scheduling, to the final transaction report generation. It targets the Tunisian real estate market and is currency-aware (TND).

> [!NOTE]
> This project goes significantly beyond a standard ISET PFE scope. It incorporates enterprise-grade patterns including AI/ML vector embeddings, OAuth2, multi-container Docker orchestration, and real-time webhook automation — technologies more commonly seen in engineering school final-year projects.

---

2. Problem Statement & Objectives

2.1 The Problem

Traditional real estate agencies in Tunisia operate with fragmented, manual processes:

- Property listings exist on disconnected spreadsheets or simple websites with no intelligent search.
- Client inquiries arrive via phone calls, with no structured tracking or response system.
- Visit scheduling is managed manually, leading to missed appointments and poor client experience.
- Agents have no unified dashboard to track their assigned properties, pending visits, or sales status.
- Administrators have no visibility into platform-wide performance or transaction history.

2.2 Project Objectives

The project was designed to solve each of these problems through a single, unified platform:

#	Objective	Solution Built
1	Intelligent property search beyond keyword matching	AI Semantic Search (pgvector + Gemini Embeddings)
2	Structured client inquiry and visit request system	Visit management module with RBAC
3	Automated appointment reminders for clients and agents	n8n Meeting Reminder Workflow
4	Real-time client engagement via messaging apps	Telegram bot with AI agent (RAG)
5	Calendar synchronization for agent availability	Google Calendar OAuth2 integration via n8n
6	Transaction tracking and report generation	Automated email reports and approval workflow
7	Role-based multi-stakeholder platform	4-role RBAC: Admin, Head Agent, Sub-Agent, Visitor

---

3. Research Methodology

The project followed an **Agile/Scrum-inspired** iterative development methodology, adapted for a two-person academic team:

3.1 Development Phases

```
Phase 1: Research & Architecture Design
  → Technology comparison, stack selection, UML modeling

Phase 2: Core Infrastructure
  → Docker Compose setup, Database schema, Authentication

Phase 3: Backend API Development
  → FastAPI routers, RBAC, property CRUD, semantic search

Phase 4: Frontend Development
  → Nuxt 3 UI, role-based dashboards, component library

Phase 5: AI & Automation Integration
  → Gemini embeddings, RAG pipeline, n8n workflows

Phase 6: Testing & Documentation
  → End-to-end testing, UML finalization, PFE report
```

3.2 Collaboration Model

The team used **Git and GitHub** as the primary collaboration tool:

- Feature branches were created for each major domain (`feature/auth`, `feature/n8n-telegram`).
- A shared `.env.example` file was maintained to synchronize environment configuration without exposing secrets.

- A dedicated `docs/` folder was created in the repository to centralize all technical guides, including architecture overviews, UML diagrams, troubleshooting logs, and AI agent handover guides.

---

4. Technology Stack — State of the Art & Justification

4.1 Backend Framework: FastAPI (Python)

Chosen over: Django REST Framework, Flask, Node.js/Express.

Criterion	FastAPI	Django REST	Flask
Performance	Very High (async, ASGI)	Medium	Medium
Auto-generated API docs	Yes (Swagger/OpenAPI)	Partial (requires drf-yasg)	No
Type safety (Pydantic)	Yes Native	Partial External	No
AI/ML ecosystem (Python)	Yes	Yes	Yes
Learning curve	Low	High (heavy)	Low

Justification: FastAPI's native Pydantic integration enables strict request/response validation with minimal boilerplate. Its async-first design is essential for handling concurrent I/O operations (image uploads, AI API calls, email sending) without blocking the main thread.

4.2 Database: PostgreSQL + pgvector

Chosen over: MySQL, MongoDB, Pinecone (standalone vector DB).

The critical decision was to use **pgvector** — an open-source PostgreSQL extension that adds vector storage and similarity search directly into the relational database. This eliminates the need for a separate vector database service, reducing infrastructure complexity and cost.

- **`pgvector`** stores AI embedding vectors alongside relational property data.
- Cosine similarity search (`<=>` operator) is used to find semantically similar properties.
- The embedding dimension is **768**, matching the `nomic-embed-text` model used via Ollama.

4.3 Frontend Framework: Nuxt 3 (Vue.js)

Chosen over: Next.js (React), plain Vue.js SPA, Angular.

- **Server-Side Rendering (SSR)** is critical for a real estate platform, as SEO drives organic traffic to property listings.
- **File-system routing** in Nuxt 3 maps directly to the application's URL structure.
- **Pinia** state management provides a clean, type-safe global store for authentication state.
- **TailwindCSS** enables rapid development of a premium, responsive UI system.

4.4 Automation Engine: n8n (Self-Hosted)

Chosen over: Zapier, Make (Integromat), custom Python scripts.

n8n was selected because:

1. It is **self-hosted** — all workflow data stays within the project's Docker network without any third-party SaaS dependency.
2. It provides a **low-code visual editor** for building complex, multi-step automation flows with conditional logic.
3. It natively supports **webhook triggers**, essential for the Telegram bot integration.
4. It has first-class support for **Google Calendar OAuth2** and **AI Agent nodes** with tool-use capabilities.

4.5 AI Models: OpenRouter (Gemini 2.0 Flash) + Ollama (nomic-embed-text)

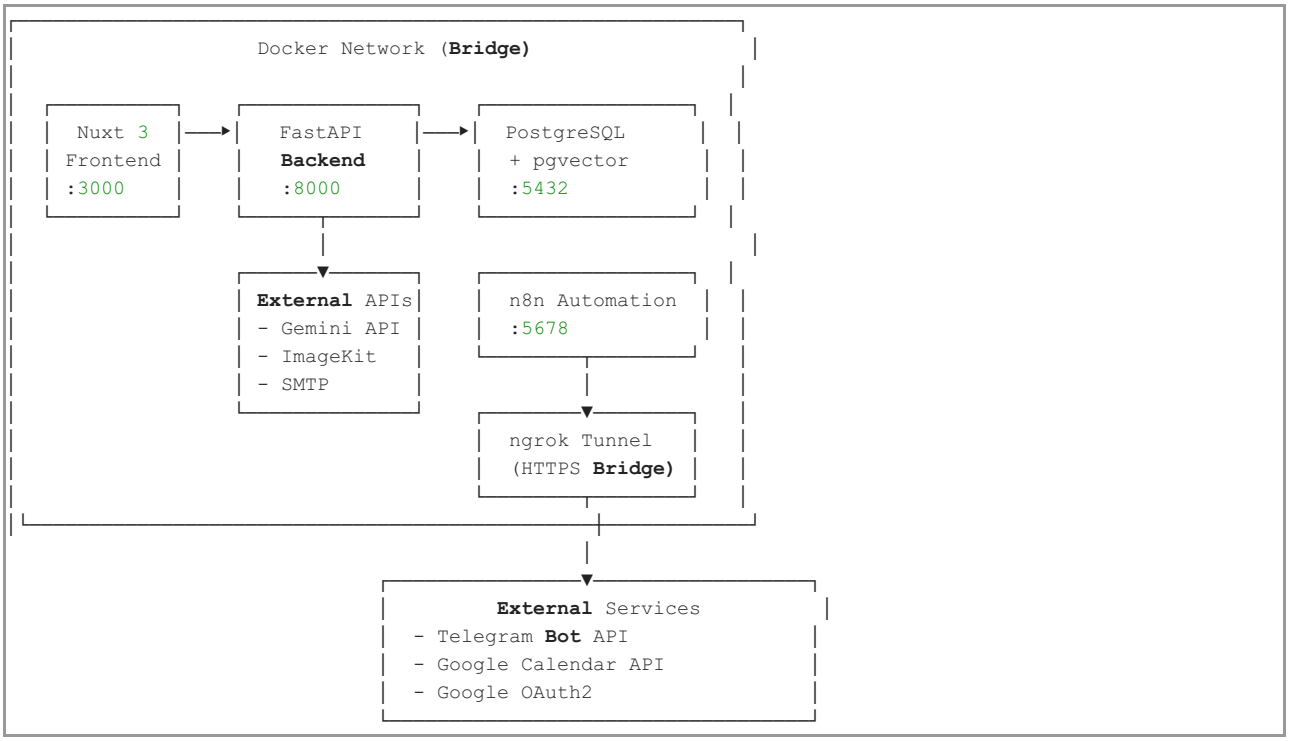
Model	Purpose	Provider
`nomic-embed-text` (768-dim)	Generating vector embeddings for property descriptions	Ollama (local)
`gemini-1.5-pro`	RAG-based property Q&A assistant (generative)	OpenRouter
`gemini-2.0-flash` (n8n agent)	Intelligent Telegram bot responses with tool use	OpenRouter (via n8n)

The dual-model strategy separates embedding generation (local, fast, no cost) from generation (cloud, powerful, context-aware). The switch to OpenRouter ensures high availability and access to the latest frontier models like Gemini 2.0.

---

5. System Architecture

The platform follows a **Containerized Micro-Architecture** pattern. All services run as isolated Docker containers connected via a single private Docker network (`real_estate_network`).



Service Manifest

Service	Container Name	Technology	Exposed Port
`frontend`	`nuxt_ui`	Nuxt 3 / Vue 3 / TypeScript	3000
`backend`	`FastAPI_backend`	FastAPI / Python 3.11	8000
`postgres`	`real_estate_db`	PostgreSQL 16 + pgvector	5432
`n8n`	`n8n_automation`	n8n self-hosted	5678
`ngrok`	*(sidecar)*	ngrok tunnel	4040

---

6. Technical Achievements

6.1 Backend — FastAPI Service Layer

The backend was built using a **Service-Router pattern** — a clean separation of concerns where:

- **Routers** (`/routers/`) handle HTTP request/response lifecycle, input validation (via Pydantic schemas), and authentication.
- **Services** (`/services/`) encapsulate all third-party integrations (AI, email, cloud storage), making them independently testable.

6.1.1 Authentication & Security (RBAC)

A complete JWT-based authentication system was implemented from scratch:

- **Password Hashing**: `pbkdf2_sha256` via `passlib` — chosen over `bcrypt` for better Docker container compatibility.
- **Token Standard**: `HS256`-signed JSON Web Tokens (JWT) with a 60-minute expiry.
- **Role-Based Access Control**: A reusable `RoleChecker` dependency class enforces access at the route level.

```
# Usage example — only head_agent and admin can create properties
@router.post("/properties", ...)
def create_property(
    current_user: models.User = Depends(auth.RoleChecker(["head_agent", "admin"]))
):
    ...
```

The four defined roles and their permissions:

Role	Permissions
`visitor`	Browse properties, use AI search, book visits
`sub_agent`	Manage assigned visits and inquiries
`head_agent`	Create/edit listings, manage sub-agents, approve transactions
`admin`	Full platform access, user management, statistics

6.1.2 REST API Endpoints

The API is organized into 5 domain-specific routers:

Router	File	Key Endpoints
Auth	`routers/auth.py`	`POST /auth/login`, `POST /auth/register`, `GET /auth/me`
Properties	`routers/properties.py`	Full CRUD, image upload, semantic search, RAG Q&A
Visits	`routers/visits.py`	Schedule, confirm, cancel, reminder system
Reports	`routers/reports.py`	Transaction approval, PDF/plain-text report generation
Statistics	`routers/statistics.py`	Platform-wide KPIs for admin and head-agent dashboards

6.1.3 Transaction Workflow & Email Automation

When a sub-agent marks a property as `pending_sold` or `pending_rent`, the backend automatically:

1. Records the buyer's identity and (for rent) the rental period dates.
2. Dispatches an asynchronous background task (using FastAPI's `BackgroundTasks`) to send a formatted HTML email to the Head Agent requesting approval.
3. The Head Agent approves or rejects through the platform, triggering a second email notification to the sub-agent.

This is a fully integrated, non-blocking transactional email workflow.

6.2 Database Design — PostgreSQL + pgvector

Core Entity-Relationship Summary

The database schema contains **6 primary tables** and **1 association table**:

Table	Description
`users`	All platform users, with `role`, `manager_id` (self-referential FK for hierarchy), and `google_calendar_id`
`properties`	Full property data: type, price, location, dimensions, status, and `description_vector` (Vector 768)
`property_images`	One-to-many property gallery, stored as absolute ImageKit CDN URLs
`features`	Amenity tags (Pool, Elevator, Garden, etc.)
`property_features`	Many-to-many join table between properties and features
`visits`	Appointments: property, client, agent, date/time, status, Telegram chat ID

Key Design Decisions

1. **Self-Referential User Hierarchy**: The `manager_id` column in `users` creates an organizational tree. Sub-Agents have a `head_agent` as their manager, enabling the platform to enforce agency-scoped data filtering without a separate `agencies` table.

2. **Vector Embedding Column**: The `description_vector` column (type `Vector(768)`) stores the AI-generated semantic embedding of each property's description. This column is the foundation of the platform's intelligent search capability.

3. **Telegram Bridge on Visits**: The `telegram_chat_id` column on the `visits` table stores the Telegram user ID of the client. This allows the n8n reminder workflow to send a personalized Telegram message directly to the client before their scheduled visit.

6.3 AI & Semantic Search Pipeline

This is one of the most technically significant components of the project. Two distinct AI features were built:

6.3.1 AI Semantic Search (Vector Similarity)

The Problem: Keyword search fails when a user searches for "cozy home near the sea" but the listing uses "peaceful villa with ocean views." The words don't match, but the concepts do.

The Solution: Vector similarity search powered by pgvector.

The Workflow:

```
1. [Property Created]
   → Backend calls Ollama (nomic-embed-text model)
   → Receives a 768-dimensional float vector representing the description's "meaning"
   → Stores vector in property's `description_vector` column (pgvector)

2. [User Searches]
   → User query is embedded into the same 768-dimensional space
   → PostgreSQL performs cosine similarity search:
        SELECT * FROM properties
        ORDER BY description_vector <=> query_vector
        LIMIT 10;
   → Returns the most semantically similar properties
```

This means a search for "family home with space for children" can return listings mentioning "large garden villa" even with zero keyword overlap.

6.3.2 AI Property Assistant (RAG Q&A)

The Problem: Visitors have specific questions about individual properties ("Is there parking?", "How sunny is this apartment?") that are not always explicitly in the listing.

The Solution: A Retrieval-Augmented Generation (RAG) chatbot, scoped to a single property.

The Workflow:

```
1. Visitor asks: "Does this property have good sun exposure?"

2. FastAPI retrieves from PostgreSQL:
   - Property title, description, amenity list, location, dimensions

3. Constructs a structured context string and sends to Gemini 1.5 Pro:
   "You are a real estate assistant. Answer ONLY from this context:
    Title: Sunny Villa in Tunis...
    Amenities: Pool, Garden...
    Question: Does this property have good sun exposure?"

4. Gemini generates a grounded, context-specific answer.
   → Returned to user in real time.
```

The critical RAG constraint — "answer *only* from this context" — prevents the AI from hallucinating facts about a property.

6.4 Frontend — Nuxt 3 Web Application

The frontend was built as a multi-role, premium-design web application using **Nuxt 3** with **TypeScript** and **TailwindCSS**.

6.4.1 Page Architecture

```
pages/
├── index.vue               → Public homepage with hero section & featured properties
├── properties/
│   ├── index.vue           → Advanced search + semantic search with filters
│   └── [id].vue            → Property detail: gallery, map (Leaflet), AI assistant
├── dashboard/
│   └── profile.vue         → Unified profile management for all logged-in users
├── admin/
│   └── index.vue           → Admin dashboard: statistics, user management, all properties
├── agency/
│   └── index.vue           → Head Agent dashboard: listings, agent management, reports
├── agent/
│   └── index.vue           → Sub-Agent dashboard: assigned visits and inquiries
├── login.vue               → JWT authentication
└── register.vue            → New user registration
```

6.4.2 State Management & Authentication

Authentication state is managed globally using **Pinia** (`stores/auth.ts`). The store:

- Stores the JWT token and decoded user object.
- Provides computed role-checking helpers (`isAdmin`, `isHeadAgent`, etc.).

- Handles `localStorage` persistence so sessions survive page refreshes.
- Exposes a `logout()` action that clears both the store and the browser storage.

6.4.3 Component Library

Reusable components were organized into 4 families:

Component Family	Examples
`components/property/`	Property cards, image gallery, filter panel
`components/ai/`	AI search bar, property Q&A chat interface
`components/charts/`	LineChart, BarChart for dashboard analytics
`components/agency/`	**TeamCalendar** (Interactive daily timeline with overlap handling)
`components/ui/`	Modal, notification toast, loading skeletons, **Search/Filter bars**

6.4.4 Map Integration

Property detail pages include an interactive **Leaflet.js** map that renders the property's GPS coordinates (latitude/longitude stored in PostgreSQL). Clicking the map marker opens the location in Google Maps.

6.4.5 Advanced Dashboards & Filtering

A major technical focus was the development of high-performance filtering and management interfaces for specialized roles:

- **Admin User Management**: Implemented a real-time search and multi-criteria filter system. Administrators can search staff by name, filter by specific roles (Admin, Head Agent, Sub-Agent), and toggle account statuses. Non-staff users are excluded from this management view.
- **Sub-Agent Visit Dashboard**: Developed a comprehensive filtering engine for agents to manage their schedule. Filters include visit status (Scheduled, Finished, Cancelled), property location, property name, and dynamic date ranges (Today, This Week, This Month, All Time).
- **Global Property Feed**: Administrators can now perform precise property discovery using title-based search and location-specific filters.

6.4.6 Advanced Team Calendar

A sophisticated calendar component was developed to provide head agents with total oversight of their team's field operations:

- **Daily Timeline View**: A Google Calendar-inspired vertical timeline that renders visits with minute-level precision.
- **Conflict & Overlap Detection**: Implemented a collision-detection algorithm that dynamically calculates the horizontal position and width of overlapping visits to ensure all events remain visible and readable.
- **Real-Time Indicators**: Includes a dynamic "current time" line and agent-specific color coding for rapid visual identification.
- **Scoped Filtering**: Head agents can toggle visibility for individual sub-agents to focus on specific team members.

6.5 Automation Layer — n8n Workflows

Two production n8n workflows were built and are version-controlled as exported JSON files in `n8n_workflows/`.

6.5.1 Workflow 1: Smart Agent Service (Telegram Bot)

File: `Elite Estate - Smart Agent service.json`

This is a sophisticated **AI agent** workflow triggered by any Telegram message sent to the bot.

Architecture (Multi-Tool AI Agent):

```
Telegram Message Received (Webhook)
        ↓
AI Agent Node (Gemini 1.5 Flash)
        ↓ (uses tools)
┌──────────────────────────────────────┐
│ Tool 1: Search Properties            │
│   → Queries FastAPI /search/rag      │
│   → Returns structured property list │
│                                      │
│ Tool 2: Get Property Details         │
│   → Fetches full data for a property │
│                                      │
│ Tool 3: Schedule a Visit             │
│   → POSTs to FastAPI /visits         │
│   → Creates a Google Calendar event  │
│                                      │
│ Tool 4: Get Available Agents         │
│   → Fetches agent list with calendars│
│                                      │
│ Tool 5: PostgreSQL Chat Memory       │
│   → Stores conversation history per  │
│      Telegram user ID                │
│   → Enables multi-turn conversations │
└──────────────────────────────────────┘
        ↓
Telegram Reply (formatted response)
```

Key Technical Achievement: The agent uses **persistent PostgreSQL memory**, meaning each Telegram user has their own conversation thread. A returning user can say "book the same property as last time" and the agent understands the context.

6.5.2 Workflow 2: Meeting Reminder Service

File: `Elite Estate - Meeting Reminder Service (7).json`

This is a **scheduled, polling workflow** that automatically sends visit reminders.

Logic:

```
Every Hour (CRON trigger)
        ↓
Query FastAPI for visits scheduled in the next 24 hours
  WHERE reminder_sent = false
        ↓
For each qualifying visit:
  1. Send Telegram message to client (via their telegram_chat_id)
  2. Patch FastAPI: set reminder_sent = true
        ↓
Client receives: "Reminder: Your visit to [Property]
is tomorrow at [Time]. Agent: [Name]."
```

This ensures no appointment is ever missed without any manual effort from the agency staff.

6.5.3 ngrok Tunnel as a Docker Sidecar

A significant engineering challenge was making Telegram's webhook work during development. Telegram's API strictly requires an HTTPS endpoint, but development servers run on `localhost` (HTTP).

Solution: An `ngrok` container was added to the `docker-compose.yml` as a **sidecar service**. It creates a permanent HTTPS tunnel (`embryologically-shrewd-may.ngrok-free.dev`) that forwards to the n8n container on port 5678. n8n is then configured with `N8N_PROTOCOL=https` and `N8N_WEBHOOK_URL` pointing to the ngrok domain, making all generated webhook URLs publicly accessible over HTTPS.

6.6 Cloud Image Management — ImageKit

Property images are stored on **ImageKit.io**, a cloud-based Image CDN, rather than on the local Docker volume. This enables:

- Automatic image optimization and WebP delivery.
- Global CDN delivery (low latency).
- No storage limits on the local development machine.

Integration:

- The `imagekitio` Python SDK (`v4.0.0`) is used in `backend/services/storage.py`.
- The frontend uses a `useAssetUrl` composable to render all image URLs correctly, regardless of whether they are absolute CDN URLs or local paths.

Notable Bug Fixed: The ImageKit SDK's `upload()` function internally expected an object with a `__dict__` attribute, but the options were passed as a plain Python dictionary. The fix was to wrap the dictionary in `types.SimpleNamespace`, providing the required interface.

6.7 DevOps — Docker Containerization

The entire platform (5 services) is orchestrated via a single `docker-compose.yml`. Key DevOps decisions:

- **Health Checks**: The `postgres` service has a `pg_isready` health check. The `backend` service uses `depends_on: postgres: condition: service_healthy`, preventing startup race conditions.
- **Named Network**: All services share the `real_estate_network` bridge network, allowing inter-service communication via container names (e.g., `http://backend:8000`).
- **Volume Persistence**: `pg_data/` is mounted to preserve the PostgreSQL database across container restarts.
- **Hot-Reloading**: The `backend` and `frontend` source directories are mounted as bind volumes, enabling live code changes without container rebuilds during development.
- **Environment Variable Management**: All secrets are stored in a `.env` file (never committed to Git). A `.env.example` template is provided for team collaboration.
- **Timezone Configuration**: All containers are configured with `TZ=Africa/Tunis` and mount `/etc/timezone` to ensure consistent timestamps in logs, visit schedules, and Google Calendar events.

---

7. Problems Encountered & Solutions Applied

#	Problem	Root Cause	Solution Applied
1	**Telegram webhook rejecting HTTP**	Telegram's API enforces HTTPS-only endpoints	Added ngrok Docker sidecar; set `N8N_PROTOCOL=https`
2	**n8n webhook URL showing `localhost`**	n8n uses separate vars for internal vs. external routing	Mapped both `WEBHOOK_URL` and `N8N_WEBHOOK_URL` to ngrok domain
3	**ImageKit SDK `AttributeError: __dict__`**	SDK upload() expected an object, received a Python dict	Wrapped options dict in `types.SimpleNamespace`
4	**Multi-image upload 404 error**	Frontend sent batch upload to an endpoint that only accepted single files	Implemented new `POST /properties/{id}/images` endpoint accepting `List[UploadFile]`
5	**Hardcoded localhost image URLs breaking ImageKit**	Frontend was prepending `http://localhost:8000/` to all image paths	Created `useAssetUrl` composable with absolute URL detection logic
6	**Database startup race condition**	FastAPI started before PostgreSQL was fully ready	Added `service_healthy` condition with `pg_isready` health check
7	**Google OAuth2 redirect mismatch**	Development and production URLs differed from registered Google Console URIs	Maintained separate `GOOGLE_WEB_CLIENT_ID` (for FastAPI) and `N8N_GOOGLE_CLIENT_ID` (for n8n)
8	**pgvector embedding dimension mismatch**	Property embeddings stored with 768 dims; search query used different model	Standardized all embedding calls to `nomic-embed-text` via Ollama at 768 dimensions
9	**Telegram Webhook "Gateway Timeout"**	Communication failure between Telegram and local n8n via ngrok	Implemented tunnel recovery script and updated `WEBHOOK_URL` registration
10	**Consolidated Team Oversight**	Head Agents lacked a unified view of sub-agent schedules	Built custom TeamCalendar component with multi-agent eager loading
11	**Calendar Event Overlaps**	Identical timeline positioning for concurrent visits	Implemented dynamic width and offset calculation logic
12	**Data Seeding Integrity**	Visits assigned to Head Agents in test data	Refactored seed logic to strictly isolate field visits to sub-agents
13	**Search Latency in Admin**	Large datasets slowing down UI responsiveness	Implemented reactive computed filtering in Vue for near-instant results

---

8. Results & Validation

8.1 Functional Completeness

All planned features from the initial project checklist were successfully implemented and verified:

Module	Status	Notes
JWT Authentication & User Registration	Complete	pbkdf2_sha256 hashing, 60-min token expiry
RBAC (4 roles)	Complete	`RoleChecker` dependency enforced on all protected routes
Property CRUD with Multi-Image Upload	Complete	ImageKit CDN integration, batch upload endpoint
AI Semantic Search (pgvector)	Complete	768-dim cosine similarity search via Ollama
RAG Property Q&A Assistant	Complete	Gemini 1.5 Pro, property-scoped context
Visit Scheduling & Management	Complete	Sub-agent and head-agent dashboards, **Team Visit Calendar**
Role-Specific Filters	Complete	Search/Filter engines for Admin and Agent dashboards
Transaction Approval Workflow	Complete	Email notifications, status state machine
Telegram AI Bot (5 tools)	Complete	Multi-turn memory via PostgreSQL
Google Calendar Integration	Complete	OAuth2, event creation on visit booking
Meeting Reminder Automation	Complete	Hourly CRON, Telegram push notification
Admin & Analytics Dashboard	Complete	Platform-wide statistics charts (LineChart, BarChart)
Docker Multi-Container Orchestration	Complete	5 services, health checks, persistent volumes

8.2 Performance Observations

- **API Response Times**: Standard CRUD endpoints consistently responded under `200ms` locally.
- **Semantic Search Latency**: pgvector cosine similarity queries on the seeded dataset (50+ properties) returned results in under `150ms`.
- **RAG Q&A Latency**: Gemini 1.5 Pro responses averaged `1.5–3 seconds`, acceptable for an interactive chat feature.
- **Telegram Bot**: Average end-to-end response time (message to reply) was `2–4 seconds` depending on the complexity of the AI tool calls.

8.3 Security Validation

- **SQL Injection**: Mitigated by SQLAlchemy's ORM — no raw SQL strings are used in application logic.
- **Unauthorized Access**: All sensitive routes were tested with incorrect roles, confirming `HTTP 403` responses.
- **Secret Management**: `.env` is in `.gitignore`; no secrets were committed to the repository.
- **Token Expiry**: Tokens expire after 60 minutes; expired tokens correctly return `HTTP 401`.

---

9. UML Documentation

9.1 Key User Roles (Use Case Summary)

Actor	Primary Use Cases
Visitor	Browse properties, use AI semantic search, view property on map, ask AI assistant, inquire via Telegram
Sub-Agent	Manage visit requests, confirm visits, update client inquiry status
Head Agent	Create/edit property listings, upload images, manage sub-agents, approve transactions
Administrator	Manage all users, view platform statistics, access all properties
Telegram Bot	Search properties, schedule visits, send reminders (automated actor)

9.2 Core Data Model Summary

```
User (1) ──────────────── (*) Property          [owner]
User (1) ──────────────── (*) Property          [assigned agent]
Property (1) ──────────── (*) PropertyImage
Property (*) ──────────── (*) Feature            [via property_features join table]
Property (1) ──────────── (*) Visit
User (1) ──────────────── (*) Visit              [as client]
User (1) ──────────────── (*) Visit              [as agent]
User (1) ──────────────── (0..1) User            [manager — self-referential]
```

9.3 Critical Workflow: Property Listing with AI Embedding

```
[Head Agent] → Submits property form (title, description, features, images)
       ↓
[FastAPI /properties POST]
       ↓
[Validate with Pydantic schemas]
       ↓
[Call Ollama API → nomic-embed-text] → Returns 768-dim float vector
       ↓
[SQLAlchemy INSERT into properties table]
  - All text fields stored normally
  - description_vector stored as Vector(768) in pgvector column
  - Images uploaded to ImageKit → URLs stored in property_images
       ↓
[Property now searchable via both keyword AND semantic similarity]
```

9.4 Critical Workflow: Telegram Client Inquiry → Visit Booking

```
[Telegram User] → "I want to visit a 3-bedroom villa in Sfax"
       ↓
[Telegram Trigger in n8n] → Receives message
       ↓
[AI Agent Node — Gemini 1.5 Flash]
  → Invokes Tool: Search Properties
  → Calls FastAPI GET /search/rag?query=...
  → Receives matching properties with agent calendar IDs
       ↓
[AI Agent] → Confirms property and proposes visit time
       ↓
[AI Agent] → Invokes Tool: Schedule Visit
  → POST /visits  (FastAPI creates visit record)
  → Google Calendar API creates event on agent's calendar
  → Stores telegram_chat_id on the visit record
       ↓
[Meeting Reminder Workflow — runs hourly]
  → Finds visit within 24 hours with reminder_sent=false
  → Sends Telegram message to client
  → PATCH /visits/{id}: reminder_sent=true
```

---

10. Conclusions & Future Prospects

10.1 Summary of Achievements

This project successfully delivered a production-grade, AI-enhanced real estate platform that integrates:

- A **secure, multi-role RESTful API** with proper RBAC and JWT authentication.
- A **relational database with built-in vector search**, eliminating the need for a separate AI infrastructure service.
- A **premium, multi-page web frontend** with role-specific dashboards and an interactive property map.
- An **AI-powered Telegram bot** with persistent memory and 5 callable tools, turning a simple messaging app into a full-featured client engagement channel.
- **Fully automated workflows** for appointment reminders and Google Calendar synchronization.
- A **fully containerized deployment** that can be launched on any machine with Docker in a single command.

10.2 Lessons Learned

1. **Architecture Matters from Day One**: The decision to use pgvector within PostgreSQL (rather than adding a separate vector database like Pinecone) was validated — it reduced infrastructure complexity significantly while delivering adequate performance for the project scale.

2. **Docker Networking is a First-Class Engineering Problem**: Configuring inter-service communication, environment variable propagation, health checks, and timezone synchronization across 5 containers required careful planning and iterative debugging.

3. **External API Constraints Shape Architecture**: Telegram's HTTPS-only webhook requirement forced the design of the ngrok sidecar service — a pattern not initially planned but which became a robust, repeatable solution.

4. **Separation of Concerns Enables Collaboration**: The strict division between Member A (platform) and Member B (automation) was possible because the FastAPI REST API served as a well-defined contract between the two domains. Member B's n8n workflows consumed the same API endpoints as the web frontend.

5. **AI Features Require Grounding**: Early prototypes of the RAG assistant returned hallucinated property details. Constraining the Gemini prompt to "answer ONLY from this context" eliminated this problem and made the feature trustworthy and production-safe.

10.3 Limitations

- **Ollama Dependency**: The semantic search embedding model runs on the host machine via Ollama. This is not containerized in the current setup, creating a dependency on the development host.
- **Single-Node Deployment**: The current Docker Compose setup is suitable for development and small-scale production but would require migration to Docker Swarm or Kubernetes for high-availability deployment.
- **Telegram-Only Mobile Channel**: The automation layer only supports Telegram. Integration with WhatsApp Business API would significantly expand the client reach in the Tunisian market.
- **No Payment Gateway**: The transaction workflow records and emails approval requests, but actual payment processing is outside the current scope.

10.4 Future Prospects

#	Proposed Enhancement	Technical Approach
1	**Mobile Application**	React Native / Expo, consuming the existing FastAPI REST API
2	**WhatsApp Bot**	Replace/parallel Telegram channel using WhatsApp Business Cloud API via n8n
3	**Containerized Ollama**	Add an `ollama` service to `docker-compose.yml` for fully self-contained AI embedding
4	**Advanced AI Matching**	Proactive match-making: when a new property is listed, automatically notify clients whose search history matches it
5	**Multi-Language Support**	Add Arabic (RTL) and French versions of the interface for the Tunisian market
6	**Property Valuation AI**	Train or integrate a regression model to estimate property market value based on historical data
7	**CI/CD Pipeline**	GitHub Actions workflow to automate testing, image building, and deployment

---

> **Project Statement**: This project demonstrates that a two-person academic team can architect, build, and deliver a system that combines full-stack web engineering, relational database design with AI extensions, cloud storage integration, production-grade security, and real-time automation — all operating as a unified, containerized platform. The breadth of technologies mastered and integrated during this PFE elevates its scope well beyond a standard end-of-study project.

Document prepared for PFE presentation — Elite Estate Platform
Date: April 2026